

Hibernate & JPA

Q1. An entity's @Id field is annotated @GeneratedValue(strategy = GenerationType.IDENTITY), and the developer manually sets an id value before calling save(). What happens?

- A) Hibernate always overwrites it with a database-generated value silently and externally Hibernate ignores the entity entirely
- B) The manually set value is used only if the database allows explicit identity inserts; otherwise, behaviour is driver-dependent and error-prone
- C) The save() call throws a compile-time error
- D) Hibernate ignores the entity entirely

Q2. A field changes on a managed entity inside a transaction, but the developer never calls repository.save() explicitly. The change still persists to the database at commit. What Hibernate feature explains this?

```
@Transactional
public void updateSalary(Long id, double salary) {
    Employee e = repo.findById(id).get();
    e.setSalary(salary); // no explicit save() call
}
```

- A) This is a bug; nothing should persist without save()
- B) Dirty checking
- C) Hibernate polls the database every second
- D) The @Transactional annotation itself writes the SQL directly

Q3. A @ManyToOne association is set to FetchType.EAGER by default, and profiling shows every simple query on the parent entity also loads a large, rarely-used child collection. What is the recommended fix?

- A) Delete the association entirely
- B) Change fetch strategy to LAZY and fetch explicitly (JOIN FETCH/@EntityGraph) only when needed
- C) Switch the database vendor
- D) Add more indexes only

Q4. Two transactions read the same row, both modify it, and the second commit should fail if the row changed since it was read, to avoid lost updates. Which mechanism fits?

- A) Optimistic locking using a @Version column
- B) Pessimistic locking only and write the SQL query directly
- C) Disabling transactions
- D) Increasing connection pool size

Q5. A developer wants to map a Java enum field so the database column stores readable text ('ACTIVE', 'INACTIVE') instead of ordinal integers. Which annotation configuration is correct?

```
@Enumerated(EnumType._____)
private Status status;
```

- A) ORDINAL
- B) TEXT
- C) STRING
- D) NAME_VALUE

Q6. A composite key spans two columns (orderId, productId) in an order-line-item table. Which JPA approach is standard for this?

- A) Use two separate @Id annotations on the same entity with String @ID
- B) Use an @Embeddable key class with @EmbeddedId (or @IdClass)
- C) Concatenate both values into a single String @Id
- D) JPA does not support composite keys

Q7. A query needs to select only employee names and departments (not full entities) for a lightweight report, avoiding the overhead of loading entire entity graphs. What is the appropriate JPQL approach?

```
SELECT new com.app.dto.EmployeeSummary(e.name, e.department.name) FROM Employee e
```

- A) This syntax is invalid JPQL
- B) A DTO/constructor projection
- C) This will fail because DTOs cannot be constructed in JPQL
- D) It will load full Employee entities regardless

Q8. A bean is declared @Scope("prototype") and injected into a singleton-scoped controller as a normal field. What issue arises?

- A) Only one prototype instance is created at startup and reused, defeating the purpose of prototype scope
- B) A new prototype instance is created on every request automatically
- C) The application refuses to start with error message (e.g. Bean is not loaded in by the spring in spring container)
- D) Prototype scope is not a valid Spring scope

Q9. A configuration class defines two @Bean methods returning the same interface type, and the team wants one of them to be the default when no qualifier is specified. Which annotation marks the default?

- A) @Default
- B) @Priority(1)
- C) @Primary
- D) @Preferred

Q10. A class needs to run initialization logic (e.g. warming a cache) right after all its dependencies are injected by Spring. Which mechanism fits?

- A) A public constructor with no arguments

- B) Overriding toString() method do display the string output from the outside of class
- C) Marking the class @Deprecated
- D) Implementing InitializingBean or annotating a method with @PostConstruct

Q11. In a Spring Boot app, a REST endpoint should only be exposed when a specific property is set in application.properties, e.g. feature.newApi.enabled=true. Which annotation supports this conditional bean registration?

- A) @Profile only, with no property option
- B) @ConditionalOnProperty(name = "feature.newApi.enabled", havingValue = "true")
- C) @RequestMapping(conditional = true, @PathVariable String/integer variable) as property
- D) @Lazy

Q12. A field-injected @Autowired dependency makes unit testing hard because the field can't be set without Spring's context. Which injection style is generally preferred to improve testability?

```
// Current approach
@Autowired
private PaymentService paymentService;
```

- A) Field injection is always preferred; no change needed
- B) Constructor injection, which makes dependencies explicit and testable via plain 'new' calls
- C) Switch to static fields
- D) Use @Autowired(required = false) instead

Q13. A Spring Boot application needs to expose health checks, metrics, and environment info for monitoring in production. Which starter provides this out of the box?

- A) spring-boot-starter-web only
- B) spring-boot-starter-test
- C) spring-boot-starter-validation
- D) spring-boot-starter-actuator

Q14. A developer wants externalized, type-safe configuration binding for a group of related properties (app.mail.host, app.mail.port, etc.) into a single POJO. Which annotation is designed for this?

- A) @Value on each field individually only
- B) @ConfigurationProperties(prefix = "app.mail")
- C) @Environment
- D) @PropertySource alone, with no binding with variable

Q15. A client submits the same POST /orders request twice due to a network retry, unintentionally creating two identical orders. What REST design principle was violated by not accounting for this?

- A) POST is not idempotent by design
- B) POST should always be idempotent by default, so this cannot happen
- C) The client should never retry any request

D) This is expected and requires no design consideration

Q16. A client sends malformed JSON in a POST request body. Which HTTP status code should the server return?

- A) 200 OK
- B) 400 Bad Request
- C) 404 Not Found
- D) 301 Moved Permanently

Q17. A path variable representing a resource ID must be extracted from the URL /employees/{id}. Which annotation is correct in a Spring controller method?

```
@GetMapping("/employees/{id}")  
public Employee getEmployee(@_____ Long id) { ... }
```

- A) @PathVariable
- B) @RequestParam
- C) @RequestBody
- D) @ModelAttribute

Q18. An endpoint DELETE /orders/5 is called on an order that was already deleted previously. To keep the endpoint idempotent, what is a reasonable response?

- A) 500 Internal Server Error every time
- B) 204 No Content or 404 Not Found, consistently, rather than an error that implies something new went wrong
- C) 301 Redirect to /orders
- D) Always return 201 Created consistently, rather than an error that implies something new went wrong

Q19. A client needs to fetch only specific fields of a resource instead of the full representation, to reduce payload size on mobile. Which REST-friendly approach is common?

- A) Always return the full object regardless of client needs and return object with its serialization ID
- B) Support optional query parameters like ?fields=name,email for partial responses
- C) Force the client to parse and discard unused fields
- D) Return binary data only

Q20. A team building REST APIs wants requests and responses to include no session state on the server between calls, so any server instance can handle any request. This is the REST constraint of:

- A) Layered System
- B) Uniform Interface only
- C) Cache-Control
- D) Statelessness

Q21. A new instance of the Account Service starts up and needs other services to be able to discover and call it without hardcoded IPs. What must it do at startup?

- A) Nothing; discovery is fully automatic without configuration
- B) Register itself with the Service Registry (e.g. Eureka)
- C) Broadcast a UDP packet to all services
- D) Wait for manual IP configuration by an admin

Q22. The Loan Service calls the Account Service via Feign, and the Feign client interface method throws a generic exception on any downstream failure, making error handling in callers difficult. What is a good fix?

- A) Ignore all exceptions silently
- B) Remove Feign and use raw sockets to translate downstream errors into meaningful, specific exceptions
- C) Implement a custom Feign ErrorDecoder to translate downstream errors into meaningful, specific exceptions
- D) Catch Throwable everywhere and log nothing

Q23. During a load spike, the API Gateway needs to prevent any single client from overwhelming backend services with excessive requests per second. Which capability addresses this?

- A) Load balancing only
- B) Rate limiting at the gateway
- C) Service discovery
- D) Config refresh

Q24. A monolith is being decomposed into microservices, and the team debates how to split the single relational database. Which principle is generally recommended for microservices data ownership?

- A) Each microservice should own its own database/schema, exposing data only via its API
- b) All services should continue sharing one database and schema
- C) Databases should be merged into a single global table and it is easy to access that table globally.
- D) Data ownership is irrelevant in microservices

Q25. Requests across microservices are hard to trace end-to-end when debugging a production issue spanning five services. Which practice helps correlate logs across services for a single request?

- A) Turning off logging to reduce noise
- B) Propagating a distributed trace/correlation ID (e.g. via Spring Cloud Sleuth/Micrometer Tracing) across service calls
- C) Restarting all services after every request
- D) Using only local variable names consistently

Q26. A Notification Service should react whenever an Order is placed, without the Order Service needing to know which services care about that event. Which style fits best?

- A) Direct synchronous REST call from Order Service to every interested service
- B) Event-driven architecture
- C) Polling the Order Service database every second from each interested service

D) Hardcoding Notification Service's URL into Order Service

Q27. SonarQube reports a NullPointerException risk on a specific line where an object could be null based on static analysis, and this would cause a runtime crash. How is this typically classified?

- A) Code Smell
- B) Duplication
- C) Documentation issue
- D) Bug

Q28. A project's Quality Gate is configured to fail if any new Blocker or Critical issues are introduced, even if overall metrics look fine. A PR introduces one new Critical issue. What happens to the Quality Gate status?

- A) It passes because overall coverage is still high
- B) It fails, because the configured condition on new Critical issues was violated
- C) SonarQube ignores severity levels entirely
- D) It passes automatically after 24 hours

Q29. A method has deeply nested if/else and switch statements, making it hard to reason about all execution paths, though it isn't literal duplication. Which SonarQube metric flags this specifically?

- A) Duplicated Lines
- B) Cognitive Complexity / Cyclomatic Complexity
- C) Coverage
- D) Lines of Code only

Q30. A hardcoded database password is found directly in source code during a SonarQube scan. How is this typically flagged?

```
String password = "admin123";
```

- A) As a minor code smell with no real risk
- B) It is not detected by static analysis tools with major code smell
- C) As a Vulnerability / Security Hotspot for hardcoded credentials
- D) As a duplication issue only

Q31. A legacy module has very low test coverage, and the team wants new code going forward to be held to a higher standard without being blocked by old, untested code. Which SonarQube concept supports this?

- A) Overall project rating only, with no distinction for new code
- B) The 'New Code' period / Quality Gate conditions scoped to newly added or changed code
- C) Deleting the legacy module entirely before analysis
- D) Disabling the Quality Gate permanently

Q32. The Maintainability Rating (A–E) in SonarQube is primarily derived from:

- A) The number of contributors to the repository and test cases evaluation

- B) The technical debt ratio relative to the codebase size
- C) The number of open pull requests
- D) The programming language used

Q33. A test class is annotated to enable Mockito annotations automatically without calling MockitoAnnotations.openMocks() manually in a JUnit 5 test. Which extension is used?

```
@ExtendWith(_____.class)
class OrderServiceTest { ... }
```

- A) SpringExtension
- B) JUnitExtension
- C) TestExtension
- D) MockitoExtension

Q34. A test stubs a method to return different values on successive calls within the same test. Which Mockito syntax supports this?

```
when(repo.findStatus(1))
    ._____("PENDING")
    ._____("SHIPPED");
```

- A) thenReturn used twice, chained, to return values in sequence on consecutive calls
- B) thenReturnOnce, which only works once with chained value in sequence on the consecutive call
- C) This is not supported by Mockito at all
- D) thenAlways, which ignores call order

Q35. A test verifies that a repository's delete() method was never called during a given code path. Which line is correct?

- A) verify(repo, never()).delete(any());
- B) assertFalse(repo.delete(any()));
- C) verify(repo, atMost(0)).delete();
- D) verifyNever(repo.delete());

Q36. A test unintentionally stubs a method that is never actually invoked during the test run, and strict stubbing is enabled. What does Mockito (with MockitoExtension) do?

- A) Silently ignores the unused stub
- B) Fails the test with UnnecessaryStubbingException, flagging dead/unused stubbing
- C) Automatically removes the unused stub from the test file
- D) Marks the test as skipped

Q37. A test needs to match any non-null String argument for a method call verification, without caring about the exact value. Which matcher is correct?

- A) eq(" ")
- B) isNull()
- C) same(" ")
- D) anyString()

Q38. Mixing raw values and argument matchers in the same mocked method call causes an InvalidUseOfMatchersException. What is the fix?

```
when(repo.update(anyLong(), "active")).thenReturn(true); // mixes matcher and raw value
```

- A) This is valid and requires no change
- B) Wrap all arguments consistently using matchers, e.g. eq("active") alongside anyLong()
- C) Remove all arguments from the call entirely
- D) Use only raw values and never matchers

Q39. A test class has ten independent test methods, each needing a freshly reset mutable object beforehand, but not an expensive shared resource. Which annotation fits, run before every test?

- A) @BeforeAll
- B) @BeforeEach
- C) @AfterAll
- D) @Test

Q40. A team wants to logically group related test methods (e.g. all 'invalid input' scenarios) within one test class for readability. Which JUnit 5 feature supports this?

- A) @Tag only
- B) @Nested test classes
- C) @Disabled
- D) @RepeatedTest

Q41. A test needs to run the same logic 10 times to check for flakiness in a random-number-based function. Which annotation is designed for repeated execution?

- A) @ParameterizedTest
- B) @Test(repeat = 10)
- C) @RepeatedTest(10)
- D) @BeforeEach(times = 10)

Q42. A method under test should complete within 200ms or the test should fail. Which JUnit 5 assertion supports this directly?

- A) assertEquals(200, method())
- B) assertTimeout(Duration.ofMillis(200), () -> method())

- C) `assertTrue(method() < 200)`
- D) `assertFast(method())`

Q43. Several tests only make sense on a CI environment and should be skipped when run locally by default. Which JUnit 5 mechanism supports conditional execution?

- A) `@Disabled` unconditionally, always skipping the test but internally passed all the given test cases
- B) Deleting the test file locally
- C) There is no way to conditionally run JUnit 5 tests
- D) `@EnabledOnOs` / `@EnabledIfEnvironmentVariable` style conditional annotations

Q44. A test asserts multiple independent conditions about an object, but JUnit stops at the first failing assertion, hiding the rest. Which JUnit 5 feature reports all failures together?

- A) `assertAll(() -> assertEquals(...), () -> assertTrue(...), ...)`
- B) `assertEquals` used multiple times has no such limitation
- C) `@Test(reportAll = true)`
- D) There is no such feature in JUnit 5

Q45. A component re-renders on every keystroke in an unrelated sibling input because both share one large state object. Which pattern helps reduce unnecessary re-renders?

- A) Combine everything into one giant global state object always
- B) Split state into smaller, focused pieces (or lift/localize state) so unrelated updates don't cascade
- C) Remove `useState` and use only global variables so it retain the value of both the props in the memory
- D) Avoid using components altogether

Q46. A `useEffect` performs a subscription to a `WebSocket` and must clean it up when the component unmounts to avoid memory leaks. Which pattern is correct?

```
useEffect(() => {  
  const sub = subscribe();  
  return () => {  
    sub.unsubscribe();  
  };  
}, []);
```

- A) This is incorrect; cleanup functions are not supported in `useEffect`
- B) This correctly returns a cleanup function that React calls on unmount
- C) The empty dependency array will cause an infinite loop
- D) `subscribe()` must be called outside `useEffect`

Q47. A form has 8 input fields, each managed with its own `useState` call, resulting in repetitive code. What is a common refactor?

- A) Combine related fields into a single state object and update via a generic `handleChange` using the input's name attribute

- B) Remove state entirely and read the DOM directly on submit only but internally use four separate component with shared parent
- C) Use 8 separate components with no shared parent
- D) This cannot be simplified in React

Q48. A parent needs to pass a callback down to a child so the child can notify the parent when a button is clicked, without lifting the click handling logic itself into the child. What is this pattern called?

```
function Parent() {  
  const handleSave = () => console.log('saved');  
  return <Child onSave={handleSave} />;  
}
```

- A) Two-way binding
- B) Passing a callback prop
- C) Context API is required for this
- D) This pattern is not valid in React

Q49. Several unrelated components deep in the tree all need access to the current logged-in user without prop-drilling through every intermediate component. Which React feature is designed for this?

- A) useEffect
- B) Context API (useContext)
- C) useState in the topmost component only, passed manually
- D) Refs

Q50. A component performs an expensive calculation on every render even when its inputs haven't changed, causing noticeable lag. Which hook helps memoize the result?

```
const result = useMemo(() => expensiveCalculation(data), [data]);
```

- A) This is invalid; useMemo cannot take a dependency array
- B) This correctly memoizes the result and only recalculates when 'data' changes
- C) useMemo always recalculates on every render regardless of dependencies
- D) useEffect should be used instead of useMemo here

Hibernate & JPA

Q1: B	Q2: B	Q3: B	Q4: A	Q5: C	Q6: B
Q7: B					

Spring Core & Boot

Q8: A	Q9: C	Q10: D	Q11: B	Q12: B	Q13: D
Q14: B					

REST APIs

Q15: A	Q16: B	Q17: A	Q18: B	Q19: B	Q20: D
--------	--------	--------	--------	--------	--------

Microservices

Q21: B	Q22: C	Q23: B	Q24: A	Q25: B	Q26: B
--------	--------	--------	--------	--------	--------

SonarQube

Q27: D	Q28: B	Q29: B	Q30: C	Q31: B	Q32: B
--------	--------	--------	--------	--------	--------

Mockito

Q33: D	Q34: A	Q35: A	Q36: B	Q37: D	Q38: B
--------	--------	--------	--------	--------	--------

JUnit

Q39: B	Q40: B	Q41: C	Q42: B	Q43: D	Q44: A
--------	--------	--------	--------	--------	--------

React

Q45: B	Q46: B	Q47: A	Q48: B	Q49: B	Q50: B
--------	--------	--------	--------	--------	--------